OOP 1 (C++) Weekly Assignments

Each assignment is designed to help you practice and strengthen your programming skills. Assignments are due **one week** after they are issued. Be sure to test your code thoroughly, comment where necessary, and submit your work on time.

Instructions

1. Write **separate C++ programs** for each given task.
2. Ensure that your code is properly **indented and commented** to explain key steps.
3. Use **appropriate data types** for each input/output.
4. Save your files with meaningful names (e.g., units.cpp, user_input.cpp).
5. Submit source code files before the deadline.

Submission Guidelines

- Submit your .cpp files through the github.
- Ensure the code compiles and runs without errors.

Week 1 Activity: C++ Basics

Task 1: Mobile Phone Sales Receipt System

A mobile phone shop in Ruiru has introduced a computerized system to help sales attendants calculate customer purchases and print sales details accurately. The shop owner wants a simple C++ program that can capture phone sale information and display a customer receipt. Write a C++ program that performs the following tasks:

i. Declares variables to store: Customer name, Phone model purchased, Quantity bought, Price per phone. Prompts the user to enter the given details.
ii. Calculates the total sales amount using the formula: Total Sales Amount=Quantity Bought × Price Per Phone
iii. Displays a well-formatted receipt showing: Customer name, Phone model purchased, Quantity bought, Price per phone, Total sales amount.

Task 2: Driving Test Result Evaluation System

Rocky driving school wants to develop a simple C++ program to determine whether a learner driver has passed or failed the driving test. The program should help instructors record marks and automatically provide results. Write a C++ program that performs the following tasks:

i. Declares variables to store: Student name, Theory test marks, Practical test marks. Prompts the user to enter the given details.
ii. Calculates the average score using the formula: Average Score= (Theory Marks + Practical Marks)/2
iii. Displays the student name, theory marks, practical marks, average score, and whether the student has passed or failed. A student passes if the average score is 50 and above.

Week 2 Activity: Control Structures

Task1: Grading System (if-else ladder)

A secondary school wants to develop a simple C++ program to automatically assign grades based on students' exam marks. Write a C++ program that performs the following tasks:

i. Declares a variable to store student name and exam marks. Prompts the user to enter the required details.
ii. Uses an if–else ladder to assign grades based on the following criteria:

70 – 100 → A

60 – 69 → B

50 – 59 → C

40 – 49 → D

Below 40 → E

iii. Displays the student name, marks, and the corresponding grade.

Task 2: Admission Decision (Nested if)

A college in Ruiru is processing student applications for admission. A student is admitted only if they meet both academic and age requirements. Write a C++ program that performs the following tasks:

i. Declares variables to store student name, age, and exam score. Prompts the user to enter the required details.

ii. Uses nested if statements to determine admission:
   - First check if age is 18 and above
     - If true, check if score is 50 and above
     - If true → "Admitted"
     - Otherwise → "Not Admitted: Low Score"
   - If age is below 18 → "Not Admitted: Underage"

iii. Displays the admission decision clearly.

Task 3: Simple Calculator (switch statement)

A small electronics shop in Ruiru wants to build a simple calculator system for sales assistants. Write a C++ program that performs the following tasks:

i. Declares variables to store two numbers and an operator (+, -, *, /). Prompts the user to enter the values.

ii. Uses a switch statement to perform the selected operation:

'+' → Addition
'-' → Subtraction
'*' → Multiplication
'/' → Division (ensure division by zero is handled)

Result=Number1operatorNumber2

iii. Displays the result of the operation.

Week 3 Activity: Functions

Task One: Payroll System Using Functions

A small manufacturing company wants to develop a simple C++ program to calculate employee salaries in a structured way using functions. Write a C++ program that performs the following tasks:

i. Defines a function *`getEmployeeDetails()`* that prompts the user to enter employee name, basic salary, and overtime hours.
ii. Defines a function *`calculateOvertimePay()`* that calculates overtime pay using the formula: *`Overtime Pay=Overtime Hours×Rate Per Hour`*
iii. Defines a function *calculateNetSalary()* that computes net salary using: `Net Salary=Basic` *`Salary+Overtime Pay`*
iv. Defines a function *`displayPayslip()`* that prints a well-formatted payslip showing all details.
v. The main function should call all the above functions in a logical sequence.

Task Two: Water Billing System Using Functions

A water supply company wants to develop a simple C++ program to calculate monthly water bills using modular programming (functions). Write a C++ program that performs the following tasks:

i. Defines a function `getCustomerDetails()` that prompts the user to enter customer name and number of units consumed.
ii. Defines a function `calculateBill()` that computes the water bill using the formula: `Water Bill=Units Consumed × Rate Per Unit`
iii. Defines a function `applyDiscount()` that reduces the bill by 10% if the customer uses more than 100 units.
iv. Defines a function `displayBill()` that prints a full bill showing customer name, units consumed, total bill before discount, discount (if any), and final amount payable.
v. The main function should call all functions in a logical sequence to complete the program.

Week 4 Activity: Loops

Task 1: Employee Bonus System (for loop)

A company wants to automate the calculation of bonuses for its employees using a C++ program. Write a C++ program that performs the following tasks:

i. Declares variables to store employee name and basic salary.
ii. Uses a for loop to input details for 5 employees.
iii. For each employee, calculates bonus using the formula: $Bonus = 0.05 \times Basic\ Salary$
iv. Calculates total salary using: $Total\ Salary = Basic\ Salary + Bonus$
v. Displays a report for each employee showing name, basic salary, bonus, and total salary.

Task 2: Savings Withdrawal System (while loop)

A mobile money agent wants to create a simple system that allows customers to withdraw money repeatedly until they choose to stop. Write a C++ program that performs the following tasks:

i. Declares a variable to store account balance and withdrawal amount.
ii. Uses a while loop to allow repeated withdrawals as long as the balance is sufficient.
iii. After each withdrawal:
    - Deducts the amount from the balance
    - Displays the remaining balance
iv. The loop should stop when:
    - The withdrawal amount is greater than the balance, OR
    - The balance becomes zero

Task 3: Password Verification System (do-while loop)

An online system requires users to enter correct login credentials before accessing services. Write a C++ program that performs the following tasks:

i. Stores a predefined username and password.
ii. Uses a do-while loop to repeatedly prompt the user to enter username and password.

iii. Checks the input:
    - If correct → Display "`Access Granted`" and exit the loop
    - If incorrect → Display "`Incorrect credentials, try again`"
iv. The system should continue until correct credentials are entered.

Week 5 Activity: Arrays

Hotel Management System Using Arrays

A hotel chain wants to develop a simple system to monitor revenue and room occupancy. Write a C++ program that performs the following tasks:

1D Array – Weekly Revenue Tracker

The hotel records its total revenue for each day of the week (7 days).

1. Use a **1D array** `revenue[7]` to store the daily revenues.
2. Input the revenue for each day.
3. Calculate and display:
    - The **total weekly revenue**.
    - The **average daily revenue**.

2D Array – Room Occupancy (One Branch)

Each branch has **5 floors**, and each floor has **10 rooms**.

1. Use a **2D array** `occupancy[5][10]` where each element represents a room (1 = occupied, 0 = vacant).
2. Input random occupancy data (or simulate input).
3. Display the number of **occupied** and **vacant** rooms per floor.

3D Array – Multiple Branches

The hotel chain has **3 branches**, each with **5 floors** and **10 rooms per floor**.

1. Use a **3D array** **chain[3][5][10]** to represent all branches.
2. Assign random occupancy (1 or 0) to each room.
3. Calculate and display the **total number of occupied rooms across all branches**.

Week 6 Activity: Classes and Objects

Task 1: Library Book Management System (Classes and Objects)

A library wants to develop a simple C++ system to manage book records and track borrowing status using object-oriented programming. Write a C++ program that performs the following tasks:

i) Create a class named `Book` with the following attributes:

- Book title
- Author
- Number of copies available

ii) Include member functions to:

- `inputDetails()` → to input book details
- `borrowBook()` → to reduce the number of available copies when a book is borrowed
- `displayDetails()` → to display book information

iii) In the main function, create an object of the class and demonstrate:

- Input of book details
- Borrowing a book
- Displaying updated book details

Task 2: Student Fee Management System (Classes and Objects)

A school wants to automate student fee processing using a C++ object-oriented program. Write a C++ program that performs the following tasks:

i) Create a class named `Student` with the following attributes:

- Student name
- Admission number
- Fee balance

ii) Include member functions to:

- `inputStudent()` → to input student details
- `makePayment()` → to reduce fee balance when a payment is made
- `displayStatus()` → to display student details and remaining balance

iii) In the main function:

- Create an object of the class
- Allow the user to input student details
- Process a fee payment
- Display updated fee status

Week 8 Activity: Constructors and Destructors

Task 1: Bank Account System

A bank wants to develop a simple C++ program to manage customer accounts. The system should automatically initialize account details when a new account is created and display a message when the account is closed. Write a C++ program that performs the following tasks:

i) Create a class named `BankAccount` with the following attributes:

- Account holder name
- Account number
- Account balance

ii) Implement a **constructor** that:

- Initializes all data members when an object is created
- Displays a message: “Account successfully created”

iii) Include a member function `displayAccount()` to display account details.

iv) Implement a **destructor** that:

- Displays a message: “Account closed successfully” when the object is destroyed

v) In the main function:

- Create an object of the class
- Display account details

Task 2: Hotel Room Booking System (Constructors and Destructors)

A hotel wants to develop a simple C++ system to manage room bookings. Each time a room is booked, the system should initialize booking details automatically, and when the booking ends, it should display a cleanup message. Write a C++ program that performs the following tasks:

i) Create a class named `RoomBooking` with the following attributes:

- Guest name
- Room number
- Number of days booked

ii) Implement a **constructor** that:

- Initializes all attributes when an object is created
- Displays a message: “Room booking confirmed”

iii) Include a member function `displayBooking()` to show booking details.

iv) Implement a **destructor** that:

- Displays a message: “Booking ended, room released”

v) In the main function:

- Create an object of the class
- Display booking details

Week 9 Activity: Inheritance

Task 1: School Management System

A school wants to develop a C++ system to manage different types of users in the school system. The system should reduce repetition of code by using inheritance. Write a C++ program that performs the following tasks:

i) Create a base class named `Person` with the following attributes:

- Name
- Age

ii) Include a member function in the base class:

- `displayPerson()` → to display name and age

iii) Create a derived class named `Student` that inherits from `Person` and adds:

- Admission number
- Class

iv) Include a member function in the derived class:

- `displayStudent()` → to display student details together with inherited details

v) In the main function:

- Create an object of the `Student` class
- Input and display all details

Task 2: Vehicle System

A transport company wants to develop a C++ system to manage different types of vehicles. The system should use inheritance to represent shared and specific properties of vehicles. Write a C++ program that performs the following tasks:

i) Create a base class named `Vehicle` with the following attributes:

- Brand
- Year of manufacture

ii) Include a member function in the base class:

- `displayVehicle()` → to display vehicle details

iii) Create a derived class named `Car` that inherits from `Vehicle` and adds:

- Number of doors
- Engine capacity

iv) Include a member function in the derived class:

- `displayCar()` → to display all car details including inherited attributes

v) In the main function:

- Create an object of the `Car` class
- Input and display all details

Week 10 Activity: Polymorphism

Task 1: Payment System (Function Overloading – Compile-time Polymorphism)

A mobile money company wants to develop a C++ system that processes different types of payments using a single function name. The system should use function overloading to handle different payment methods. Write a C++ program that performs the following tasks:

i) Create a class named `Payment` with overloaded functions named `processPayment()` that:

- Accepts an integer amount for cash payment
- Accepts two parameters (amount and transaction fee) for card payment
- Accepts three parameters (amount, fee, and discount) for mobile payment

ii) Each function should calculate and display the final payment amount.

iii) In the main function:

- Create an object of the class
- Demonstrate all three types of payments

Task 2: Shape Drawing System (Function Overriding – Run-time Polymorphism)

A graphics software company wants to develop a C++ program that can draw different shapes using runtime polymorphism. Write a C++ program that performs the following tasks:

i) Create a base class named `Shape` with a virtual function:

- `draw()`

ii) Create derived classes:

- `Circle`
- `Rectangle`
- `Triangle`

iii) Each derived class should override the `draw()` function to display:

- "Drawing Circle"
- "Drawing Rectangle"
- "Drawing Triangle"

iv) In the main function:

- Create a base class pointer
- Use it to call the `draw()` function for each derived class

v) Demonstrate runtime polymorphism by showing different outputs depending on the object referenced.

Week11 Activity: Encapsulation

Task 1: Employee Data Protection System (Encapsulation)

A company wants to develop a C++ system to store and manage employee details securely. The system should ensure that sensitive data is protected from direct access using encapsulation. Write a C++ program that performs the following tasks:

i) Create a class named `Employee` with the following **private data members**:

- Employee name
- Employee ID
- Basic salary

ii) Include **public member functions** to:

- `setEmployeeDetails()` → to input and assign values to the data members
- `calculateSalary()` → to compute net salary after adding a 10% allowance using the formula:
  `Net Salary = Basic Salary + 0.10 × Basic Salary`
- `displayEmployeeDetails()` → to display employee information and net salary

iii) In the main function:

- Create an object of the class
- Set employee details
- Display the employee details

Task 2: Student Fee Security System (Encapsulation)

A school wants to develop a C++ program to securely manage student fee information. The system should prevent direct access to sensitive fee data using encapsulation. Write a C++ program that performs the following tasks:

i) Create a class named `Student` with the following **private data members**:

- Student name
- Admission number
- Fee balance

ii) Include **public member functions** to:

- `inputStudentData()` → to input student details
- `makePayment()` → to reduce the fee balance when a payment is made
- `displayStatus()` → to show student details and remaining balance

iii) Ensure that the fee balance cannot be accessed or modified directly from outside the class.

iv) In the main function:

- Create an object of the class
- Input student details
- Process a fee payment
- Display updated status

Week12 Activity: Abstraction

Task 1: Hospital Management System (Abstraction)

A hospital wants to develop a C++ system to manage different types of medical staff. The system should use abstraction to hide implementation details while exposing only essential operations. Write a C++ program that performs the following tasks:

i) Create an **abstract class** named `MedicalStaff` with the following:

- Protected attributes:
  - Staff name
  - Staff ID

ii) Include a **pure virtual function**:

- `calculateSalary()` → to be implemented differently by each type of staff

iii) Include a **non-virtual function**:

- `displayDetails()` → to display staff name and ID

iv) Create derived classes:

- `Doctor`
- `Nurse`

Each class should:

- Implement `calculateSalary()` differently:
  - Doctor salary includes basic salary + allowance
  - Nurse salary includes basic salary + overtime pay

v) In the main function:

- Use a base class pointer
- Create objects of both derived classes
- Demonstrate runtime behavior using abstraction

Task 2: Shape Area System (Abstraction)

A graphics design company wants to calculate areas of different shapes using a C++ program. The system should use abstraction to ensure that each shape defines its own method of calculating area. Write a C++ program that performs the following tasks:

i) Create an **abstract class** named `Shape` with:

- A pure virtual function:
  - `calculateArea()`

ii) Create derived classes:

- `Circle`
- `Rectangle`

iii) Each class should:

- Implement `calculateArea()` using the appropriate formula:
  - Circle: Area=πr2
  - Rectangle: Area=Length×Width

iv) In the main function:

- Use a base class pointer
- Create objects of each shape
- Call `calculateArea()` for each object

Week 13 Activity: C++ Files

1. A local library wants to keep track of the titles of books borrowed each day. Write a C++ program that allows the librarian to enter book titles and store each title in a text file named borrowed_books.txt.
   i. The program should not delete existing records when a new title is added.
   ii. Display a confirmation message once the title is successfully stored.

2. A university stores student examination results in a binary file named results.dat. Each record contains a student's name, registration number, and total marks. Write a C++ program that:

   i. Defines a structure to hold the student information.
   ii. Reads all student records from the binary file.
   iii. Displays the name and marks of each student on the screen.

3. A small retail shop stores daily sales transactions in a file named sales.txt. Each line contains the amount of a single transaction. Write a C program that:

   i. Reads all transactions from the file.
   ii. Calculates and displays the total sales for the day.
   iii. Ensures the file is properly closed after reading.